

Lecture 5

- CSS rule syntax

`selector[, selector]* {[prop: value;]+}`

- DOM tree and elements as targets

- Selectors

- element types
- class/id attributes
- other attributes
- pseudo-classes/elements

- Combinators for selectors

Styles, solving priorities

- Specificity, cascading, inheritance

- !important (to be avoided)

- A general guideline for writing CSS

- Box model

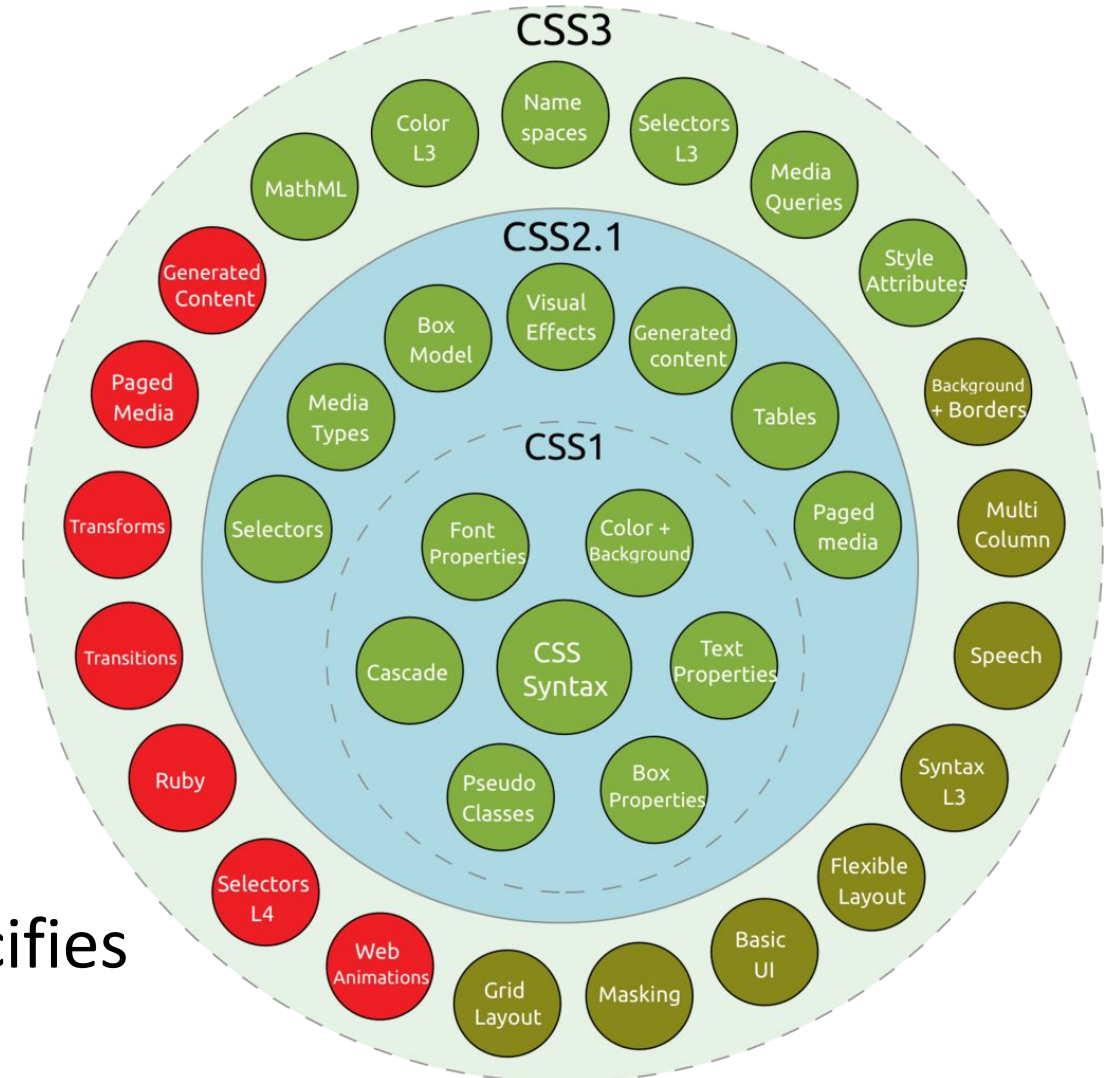
- CSS Units (absolute, relative)

- Viewport units, pixels

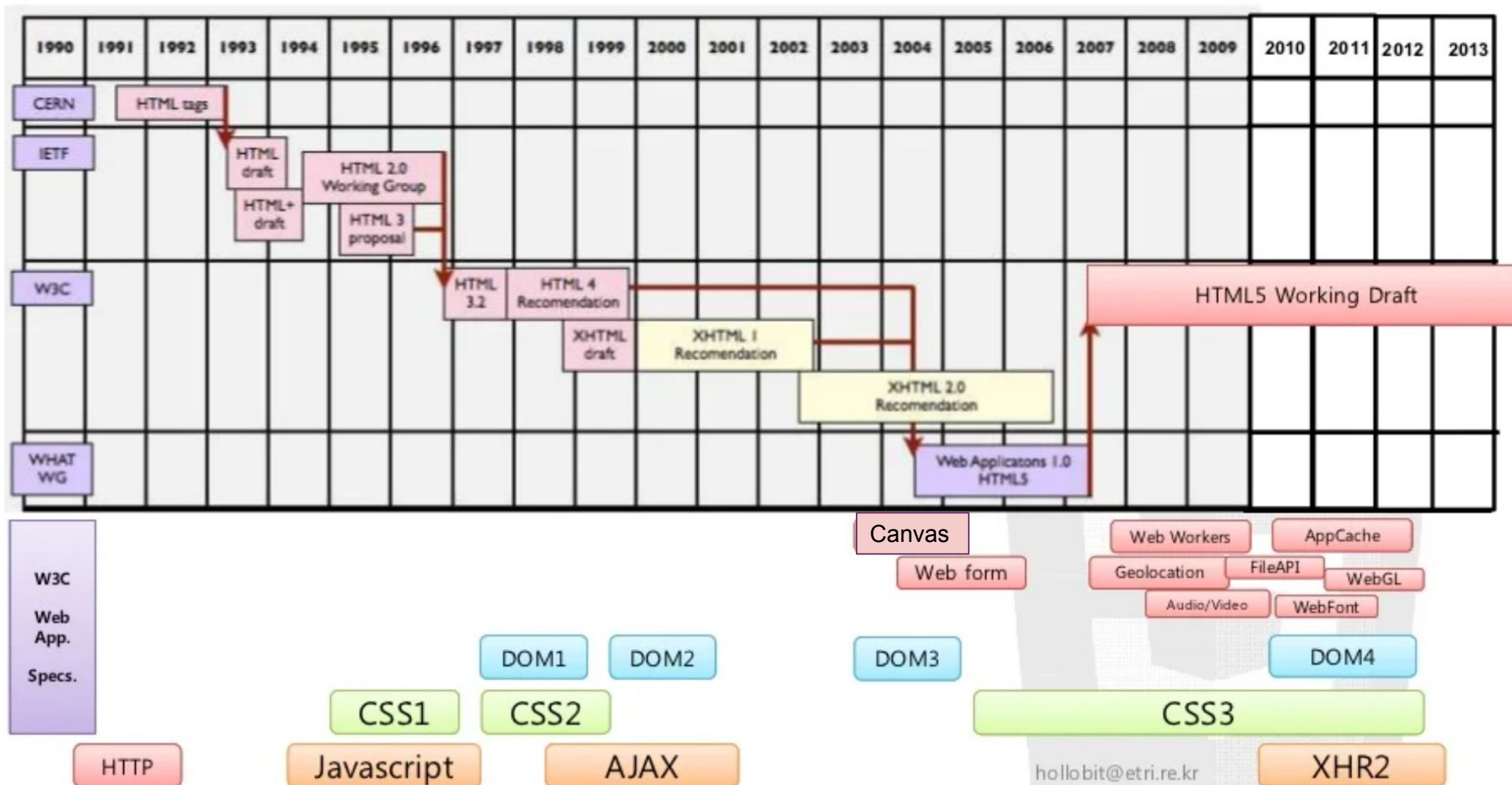
CSS, Cascading Style Sheets



- A stylesheet language used to *style* the HTML content on a web page
- Cascading refers to inheritance
 - properties are inherited within the DOM hierarchy
 - from a parent to a child
 - inheritance is property-specific, not all properties are inherited
- CSS1, CSS2, CSS2.1, CSS3 standards
- no CSS4 coming, instead, W3C specifies different independent modules



HTML5 & Web App Technology Timeline



CSS rule syntax

NOTATIONS

- *: 0 or more
- +: 1 or more
- ?: 0 or 1
- |: separates alternatives
- [: grouping

○ Syntax of a CSS rule

```
selectorList {  
    declaration  
}
```

○ The same rule targeted at several elements

```
selector[, selector]* {  
    [property: value;]+  
}
```

```
h1, h2 {          /* headers blue */  
    color: blue;  
}
```

selector

property

value

○ Several rules targeted at the same element

```
body {          /* page with black text */  
    color: black; /* on yellow background */  
    background-color: yellow;  
}
```

Example rules

- Universal selector * selects all elements

```
* { /* unifying the marginal and box sizing */
  margin: 0;
  box-sizing: border-box;
}

h1 { /* the first level header gets a black shadow */
  text-shadow: 4px 4px 2px black;
}

p { /* text in paragraphs in red, and the width */
  color: red; /* of the paragraph and the margin around it */
  width: 240px; /* are defined */
  margin: 3rem;
}
```

DOM tree structure

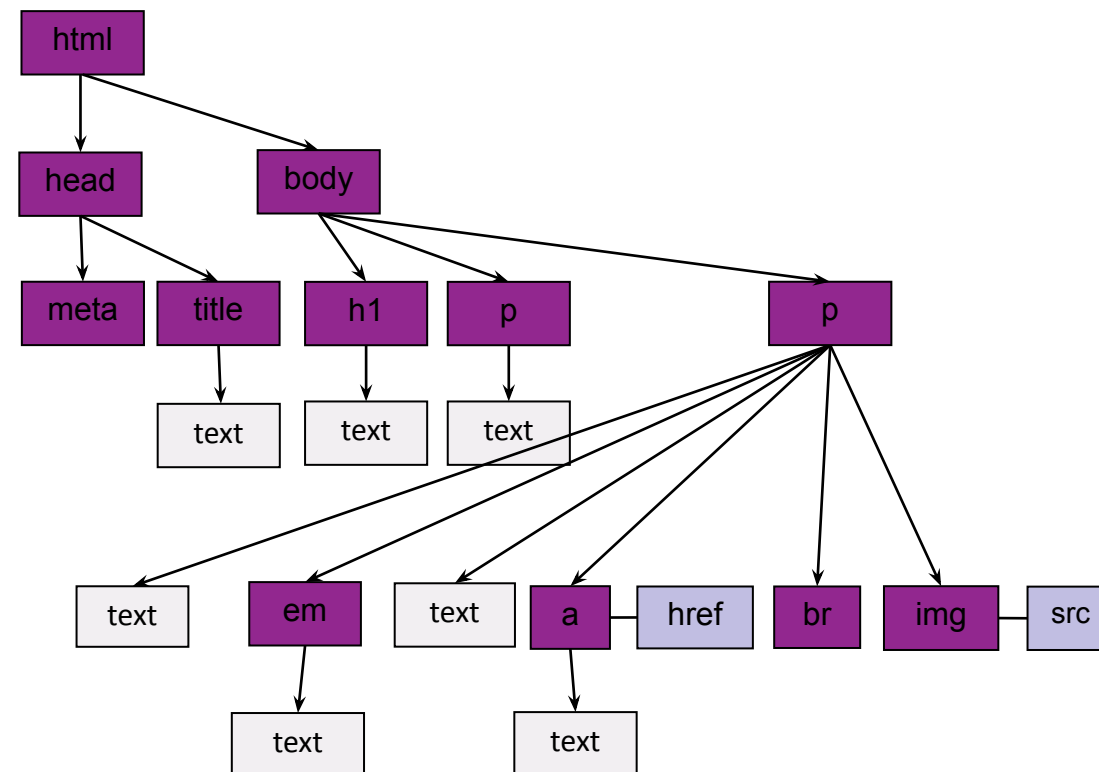
○ DOM (document object model)

- *document* comprises of hierarchy of nodes: element and text nodes

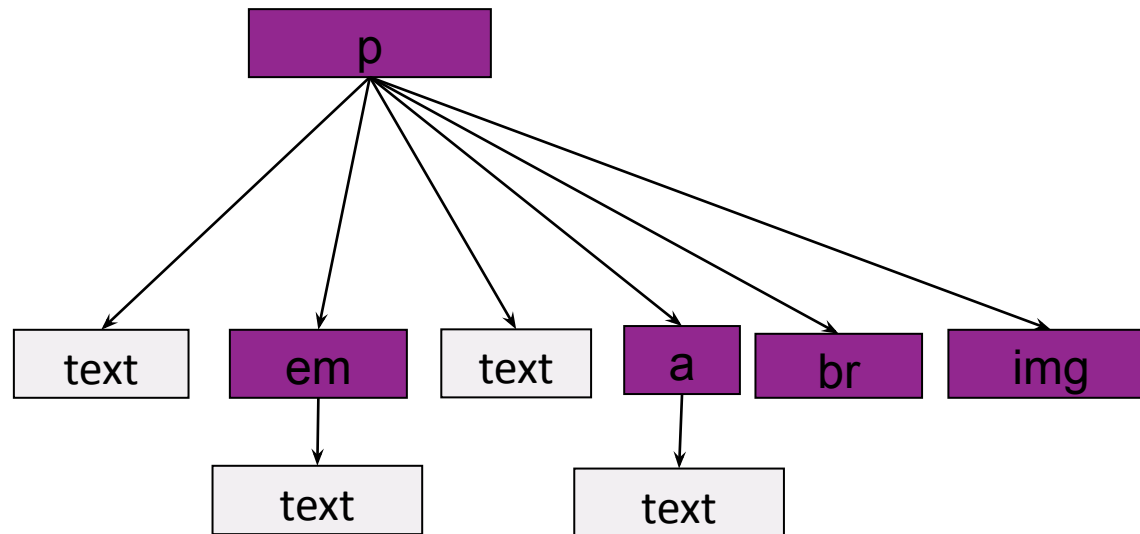
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" >
    <title>HTML page</title>
  </head>
  <body>
    <h1>
      Elements on a page form the DOM tree structure
    </h1>
    <p>
      This page contains two paragraphs
      (this is the first of them).
    </p>
    <p>
      This is <em>second</em> paragraph and here is an image from
      <a href="http://www.visittampere.fi">Tampere</a><br>
      <img src = "img/01-tampere.jpg" alt="Tampereen Hämeensilta">
    </p>
  </body>
</html>
```



in CodePen



Parent and child nodes



- **p** node has six children, **br** node has five siblings
- **text** node on the left is the first child of **p** node
- **em** node has one child (**text** node)
- **a** node is the parent of rightmost **text** node

○ DOM

- describes the structure of the HTML document as a hierarchical node tree
- important to understand when we want to modify the properties of a certain element on the page

Selectors: Element, Class, Id

- Element (type) selector
- Class selector
- Id selector

HTML-document

```
<p>
  text
</p>
```

```
<p class="important">
  text
</p>
```

```
<p id="my-name">
  text
</p>
```

CSS-rules

```
p {
  color: red;
}
```

```
.important {
  color: red;
}

p.important {
  background: red;
}
```

```
#my-name {
  color: red;
}
```


Attribute selectors

[attribute="value"]?

- Elements may be selected on the basis of their attributes and attribute values
 - text color in both paragraphs is blue, only if lang is Finnish then background will be yellow
- Another example, basic use of attribute selector
 - all the links which would open to a separate tab, are written with yellow background

```
<p lang="en">text</p>
<p lang="fi">teksti</p>
```

```
p[lang] {
  color: blue;
}
p[lang="fi"] {
  background-color: yellow
</p>
```

```
a[target="_blank"] {
  background-color: yellow
}
```

More attribute selectors

`^=` `$=` `*=` `|=` `~=`

Selector

Selects

`^=`  **`[attribute^="value"]`** all elements starting with "value" (prefixed with "value")

`$=`  **`[attribute$="value"]`** all elem. ending with "value" (suffixed with "value")

`*=`  **`[attribute*="value"]`** all elem. containing "value"

`|=`  **`[attribute|="value"]`** `E[lang|="en"]` E elements whose "lang" attribute has a hyphen-separated list of values beginning (from the left) with "en", e.g., `<span lang="en-fi">`

`~=`  **`[attribute~="value"]`** all elem. with one or more values for the attribute and *one* of them is "value"

`[attr][attr]`  **`[attr1][attr2]`** all the elem with both attributes

Pseudo-class selectors

:selector

- "pseudo" because such class does not actually exist
- Pseudo-class selector starts with a colon, p: **first-child**
- Elements are selected neither by name nor attribute, but other properties of an element, e.g., on the basis of the present status (active, checked)

:active		:active		:invalid	:only-of-type
		:checked		:lang	:optional
		:disabled		:last-child	:out-of-range
		:empty		:last-of-type	:read-only
		:enabled		:link	:read-write
		:first		:not()	:required
		:first-child		:nth-child	:root
		:first-of-type		:nth-last-child	:target
		:focus		:nth-last-of-type	:valid
		:hover		:nth-of-type	:visited
		:in-range		:only-child	



Pseudo-element selectors

::selector

- Pseudo-element selector starts with a double colon, e.g. a `::before`
- Pseudo-class selector selects elements that actually exist
- pseudo-element selector may create "pseudo-elements" that are not defined in HTML

`::after`  `::after`
`::before`

`::first-letter`  `::first-letter`
`::first-line`

Combinators

- Combinators make use of the DOM tree structure

`e1 > e2`

child combinator

selects all `e2` elements that have `e1` as their parent

`e1 e2`

descendant combinator

selects all `e2` elements that have `e1` as a parent or as an ancestor

`e1 + e2`

adjacent sibling

selects the element `e2`, if `e1` is the immediate older sibling

`e1 ~ e2`

general sibling combinator

selects all `e2` elements that have `e1` as an older sibling

```
ul > li {  
    background-color: yellow;  
}
```

```
ul a {  
    background-color: yellow;  
}
```

```
p + a {  
    background-color: yellow;  
}
```

```
li ~ li {  
    background: #ff0000;  
}
```

Chaining selectors

- By chaining selectors together you require that all the selectors match the element
- For example when an element belongs to several classes

```
<h1 class="first third">Double Class</h1>
```

```
.first.third {  
    color: red;  
}
```

- What is the difference between these two ?

```
#header.callout {  
    color:red;  
}
```

```
#header .callout {  
    color:red;  
}
```

- Or what would this mean?

```
#chapter-1.snippet.code.erroneous {  
    color: red;  
}
```

Attaching style rules to a page

Method 1

- a separate *.css -file or several files
 - attached using the <link> element

```
<!DOCTYPE html>
<html lang="fi">
<head>
  <meta charset="UTF-8" />
  <title>Sivun otsikko</title>
  <link rel="stylesheet" type="text/css" href="path/basic.css" />
  <link rel="stylesheet" type="text/css" href="path/navigation.css" />
  <link rel="stylesheet" type="text/css" href="path/special.css" />
  ...
```


Attaching style rules to a page

Method 2

- `<style>` element in the head of the HTML document
 - for example, the CSS rules to set the background color and left margin of the whole page

```
<!DOCTYPE html>
<html lang="fi">
<head>
  <meta charset="UTF-8" />
  <title>Sivun otsikko</title>
  <style>
    body {
      background-color: maroon;
      margin-left: 20px;
    }
    p { color:white; }
  </style>
  ...

```

Attaching style to an element

Method 3

- Straight into an element on a page

- using the `style`-attribute

Only the rule, no need for a selector

```
<p style="color:sienna; margin-left:20px;">  
  This is a paragraph.  
</p>
```

- Method 1 is the preferred way to do it
 - CSS-files can be shared between several pages
 - method 2 can be used for small projects
 - method 3 only for quick experimenting – never leave style attributes in the finished code

Several rules for the same element

- How to decide the layout of an element? The following three concepts should be understood.
- Specificity
 - selectors have a certain specificity value
 - the rule with the most specific selector "wins"
- Cascading (location)
 - when several rules have same specificity value, the "last" rule wins
- Inheritance
 - some properties are inherited from their parent (not all)

Specificity

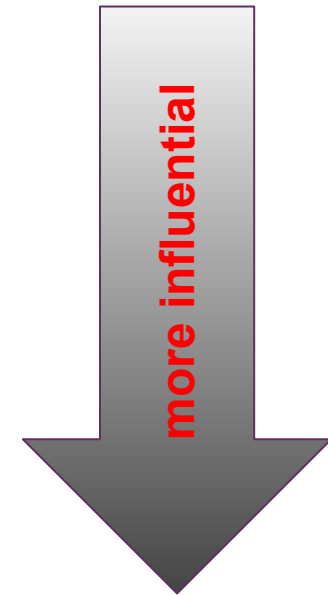
- Each selector has a specificity value as follows

Selector	Specificity
Type selectors, pseudo-element selectors (e.g. <code>p</code> and <code>::after</code>)	1
Class selectors, pseudo-class selectors, attribute selectors	10
Id-selector	100
Using style attribute within the element (inline style)	1000

- The specificity of a combined selectors are computed as a sum of the specificity values involved, e.g.
 - `p::after` - type and a pseudo-element selectors → $1+1 = \mathbf{2}$
 - `[type=email]` - an attribute selector → **10**
 - `p.important` - a type and a class selectors → $1 + 10 = \mathbf{11}$
 - `aside.chart:first-child` - a type , a class and a pseudo-class selectors → $1+10+10 = \mathbf{21}$

Cascading

- The location where the rule is given, can be
 - in the browser's default styles (user agent style)
 - in the browser's user defined styles (e.g. for Firefox via userContent.css -file)
 - in the document itself
 - in a separate CSS-file
 - in the `<style>` element in the head-part of the document
 - in the element with a `style` attribute
- A rule given on an outer level (upper in the list) cascades inwards if no rule on an inner level overrides it



Inheritance

- For example, `color` is inherited, but `border` isn't

- when you want to write the text on a page with blue
 - you give a rule for the body
 - you don't need to give it for every element
- but
 - when you give a list element (`<ul>`-element) a border line
 - it's children (`<li>`-elements) are not drawn in boxes
- however, a property can be changed to be inherited
- notice, that properties can have initial values (for color, it can be transparent)

Name:	color
Value:	<color> inherit
Initial:	depends on user agent
Applies to:	all elements
Inherited:	yes
Percentages:	N/A
Media:	visual

Name:	'border'
Value:	<line-width> <line-style> <color>
Initial:	See individual properties
Applies to:	all elements
Inherited:	no
Percentages:	N/A
Media:	visual

```
body { color: blue; }
```

```
li { border: inherit; }
```

Name:	'background-color'
Value:	<color>
Initial:	transparent
Applies to:	all elements
Inherited:	no

!important

- All the rules previously assigned to an element can be overridden by writing `!important` after a rule
- To be used in special situations only, for example
 - visually impaired users may write a rule in the user style sheet (browser file `userContent.css`)

```
body {  
  font-size: large; !important  
}
```

- Should **not** be used in normal style sheets, even if it sometimes feels tempting

A general guideline when writing CSS

- Aim to keep the selectors general
 - you can use specific selectors, but only when really needed
- CSS gets easily scattered
 - the more high-specificity selectors you use, the more difficult maintaining your stylesheet becomes

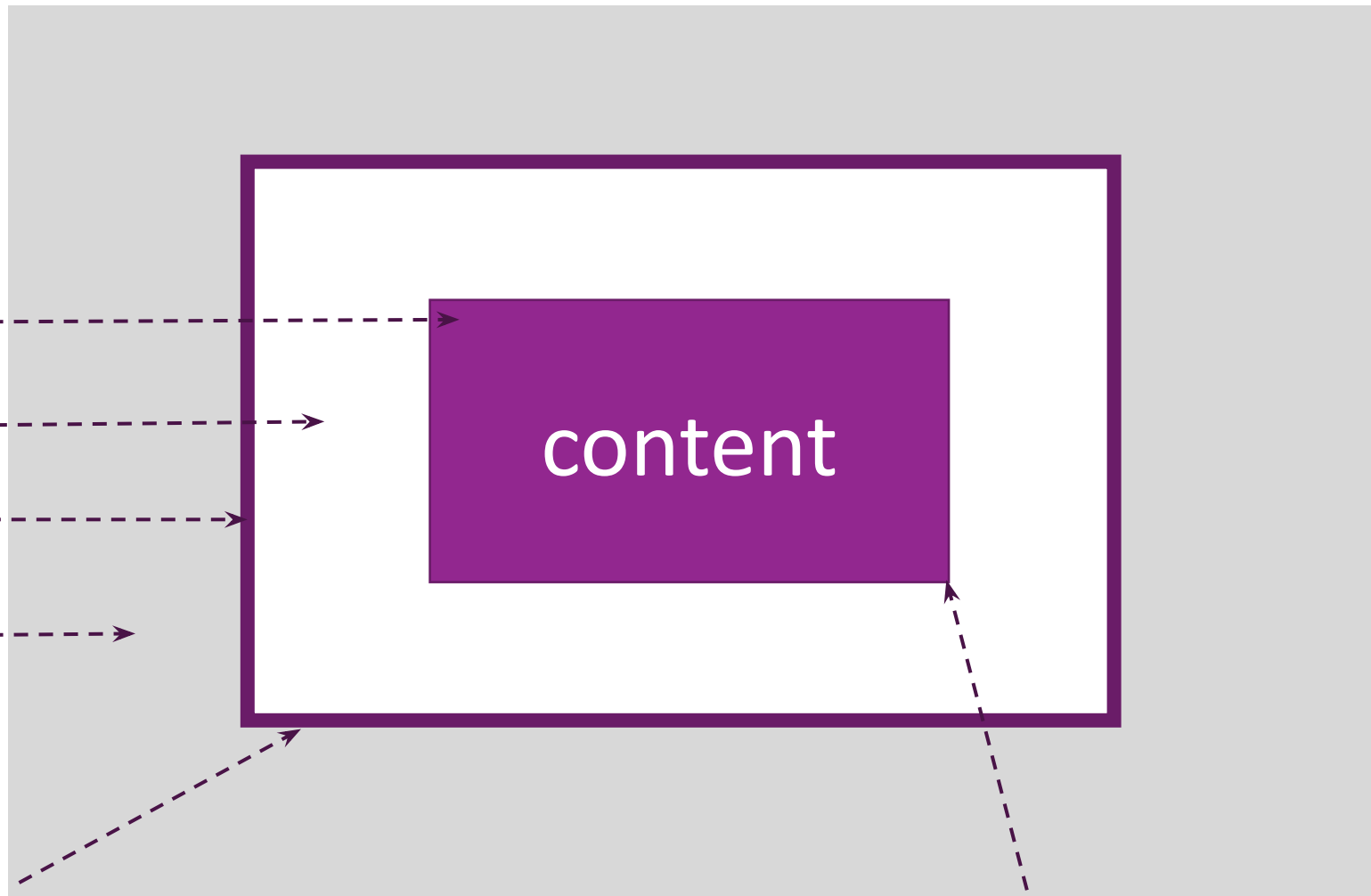
Box model

- content

- padding

- border

- margin



`box-sizing: border-box;`

`box-sizing: content-box;`

JWT

0200

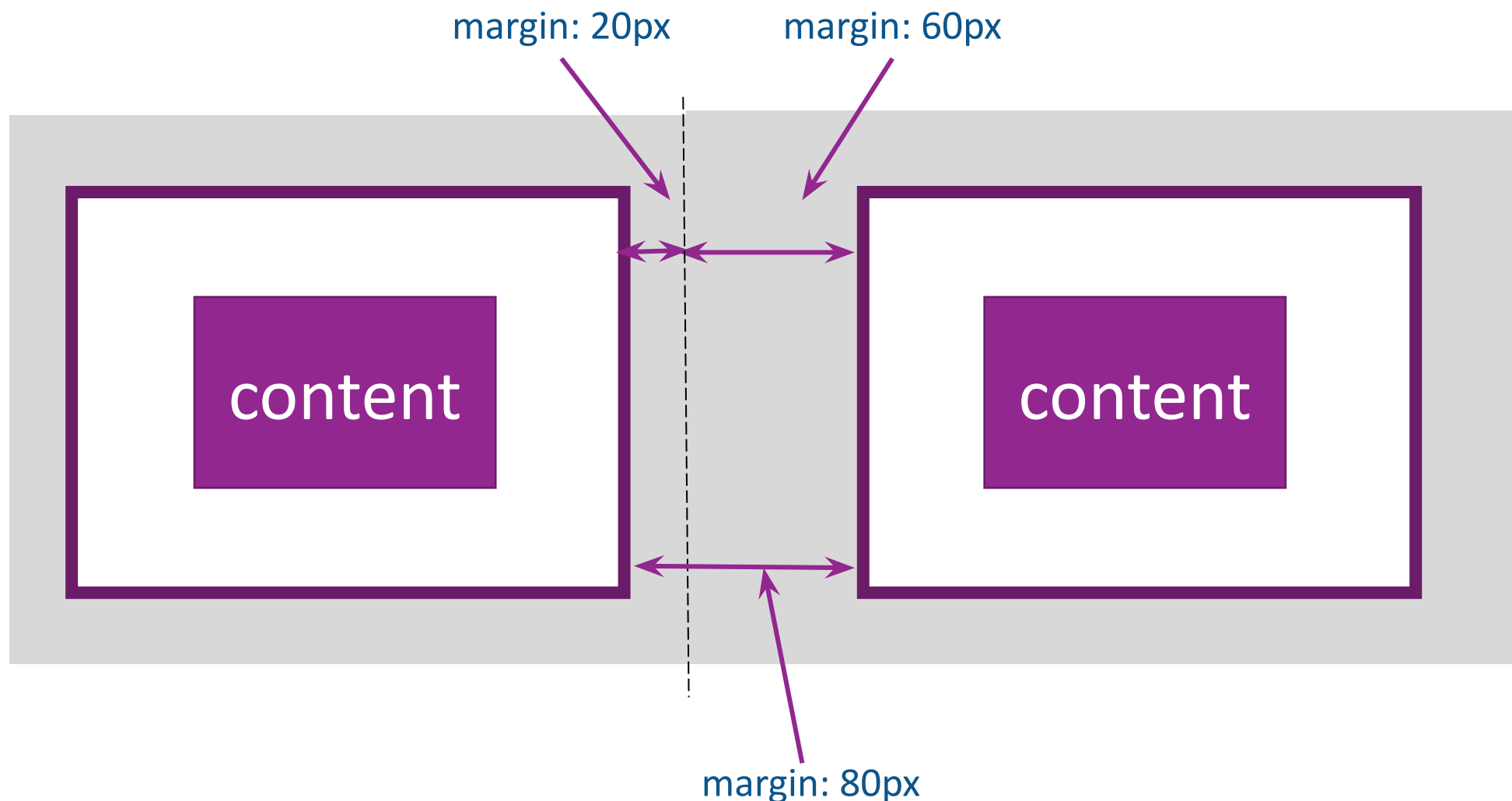
Spacing between elements: padding and margin

- Spacing between elements is handled with `padding` and `margin` properties
 - `padding: [auto|value]1-4`
 - `margin: [auto|value]1-4`
- Padding keeps the content away from the border
 - behaviour quite straightforward and predictable
 - can not be negative
- Margin leaves space between the border and other elements
 - horizontally pretty predictable
 - vertically margins of adjacent elements collapse
 - can be also negative
 - vertical placement inside the parent: `margin: auto (!)`

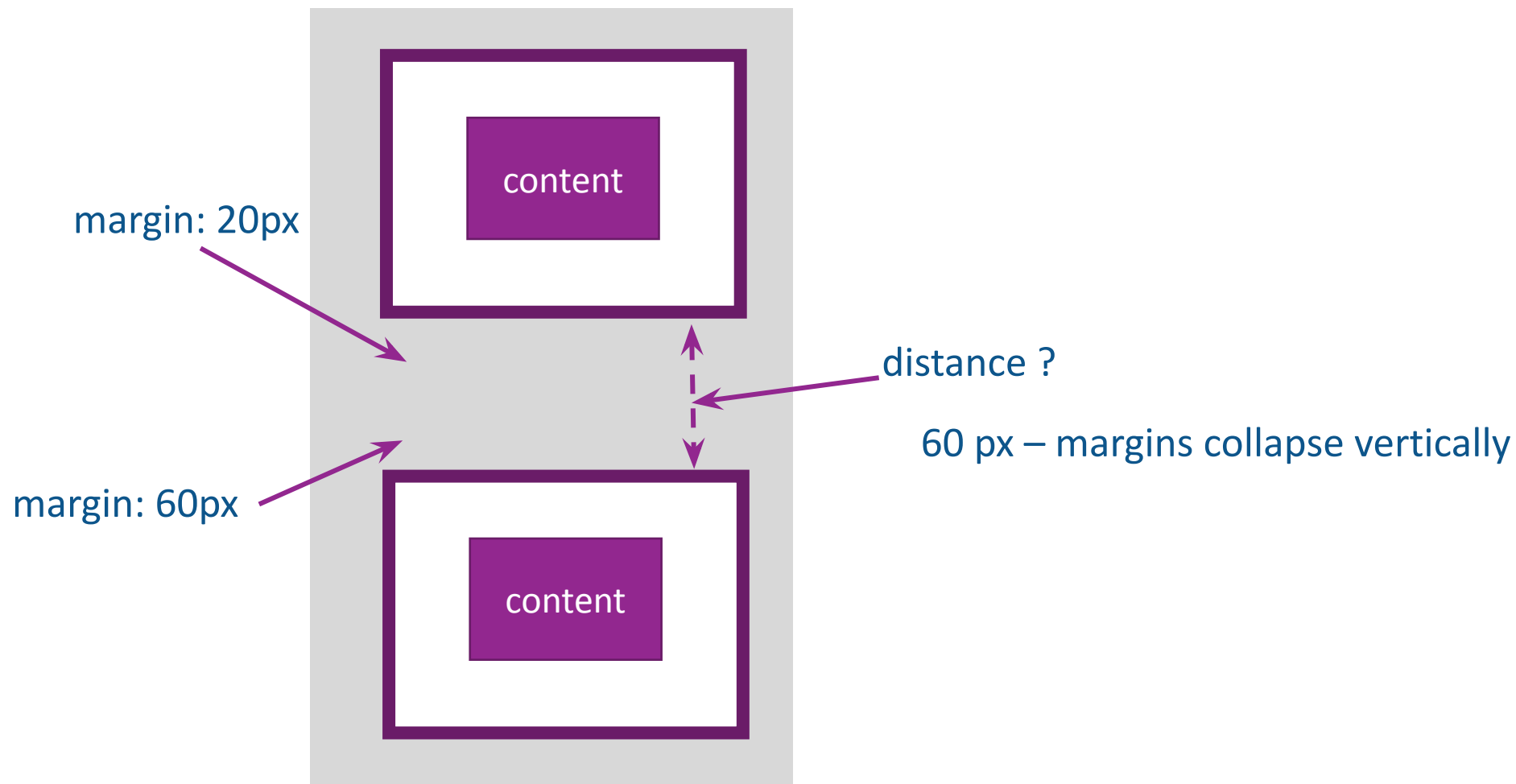


```
h1, h2 {  
  padding: 25px 50px;  
}  
  
p {  
  padding: 25px 50px 5px;  
  margin-left: 2em;  
}
```

Margin behaviour horizontally



Margin behaviour vertically



Border

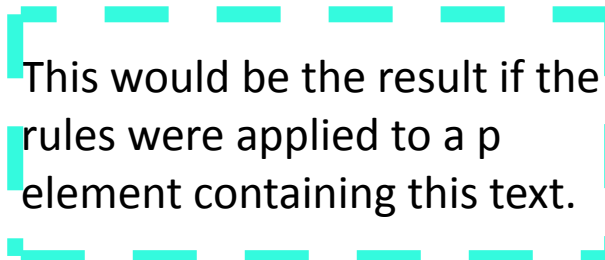
○ Border width, style, and color

- **border-width:** [thin|medium|thick|measure|inherit]¹⁻⁴
- **border-style:** [none|hidden|dotted|dashed|solid|double|inherit|groove|ridge|inset|outset]¹⁻⁴
- **border-color:** [transparent|color|inherit]¹⁻⁴

```
p {  
  border-width: 10px;  
  border-style: dashed;  
  border-color: #34fade;  
}
```



```
p {  
  border: 10px dotted #34fade;  
}
```



This would be the result if the rules were applied to a p element containing this text.



Border

The **box-sizing** property can be used to add padding and **border** in an element's width and height:

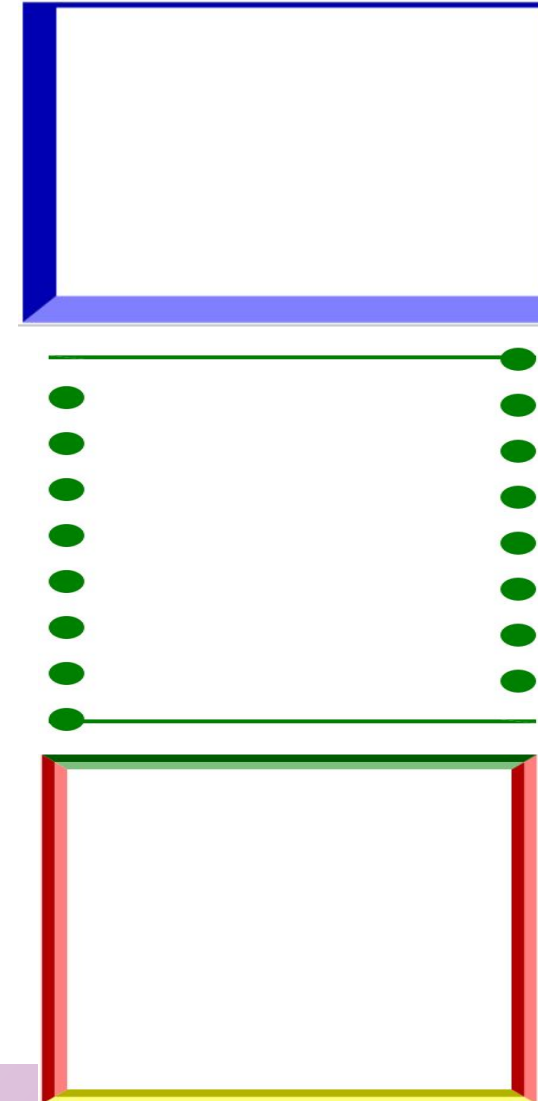
box-sizing: border-box;

Border properties

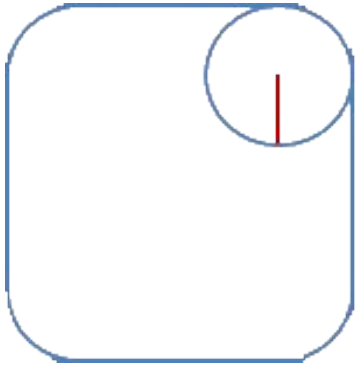
```
div {  
  border-width: 5px 15px 25px 35px;  
  border-style: inset;  
  border-color: blue;  
  width: 30em;  
}
```

```
div {  
  border-width: 5px 10px;  
  border-style: solid dotted;  
  border-color: green;  
}
```

```
div {  
  border: 20px groove;  
  border-color: green red yellow;  
}
```



Rounded corners



```
.framed-element {  
  border-radius: 10px;  
}
```

```
.framed-element {  
  border: 10px double orange;  
  border-radius: 5px 50px 50px 50px;  
}
```



CSS units (of length measure)

- Used to define lengths and sizes in CSS rules
- The **absolute units**: in, cm, mm, pt, pc
 - not useful on screen, might get used on some other medium
- Pixel (px) is a very commonly used one, e.g.,
 - `min-width: 200px; font-size: 16px;`
- In addition to absolute, many units are relative
 - relative with respect to parent element or to the viewport (= visible part of the page)
- Relative units include: %, em, rem, ex, ch, vh, vw
 - note: no space between the number and the unit, e.g.
16px, 0.75em, 1.25rem, 80%, 90vh, 85vw

Relative units

- %**
 - relative to parent dimensions, handy when the parent's dimensions are set
 - however often tricky in the vertical dimension

if parent element doesn't have an explicit height set, the child percentage is always 0
- em**
 - relative to its parent's font-size (height of the font)
 - usually the default font-size (for the `<body>`- element) in browsers is 16px
- rem**
 - relative to the font size of the root element
- ex**
 - height of the letter "x" (small x) in the used font
- ch**
 - the width of the number "0" in the used font
-

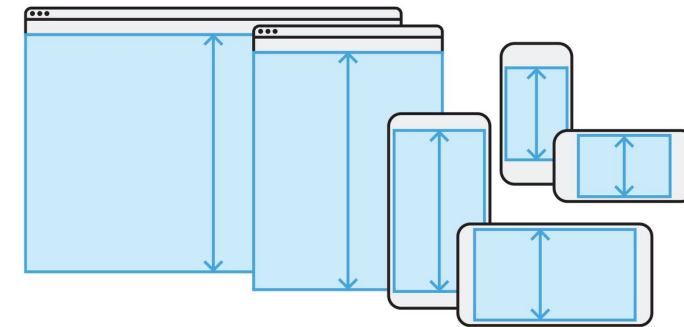
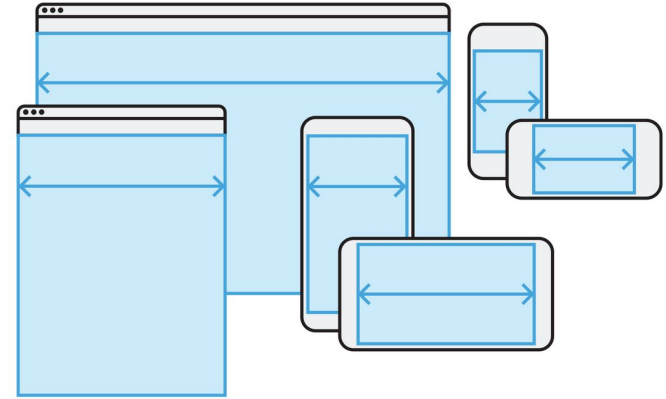
Viewport units, pixels

vh / vw

- relative, like %-unit
- instead of percentage of the parent, it is percentage of the screen (aka viewport)
- `vh` and `vw` do not suffer from the parent limitation that the %-unit does
- `vh` refers to one percent of viewport height, and `vw` to one percent of viewport width

- px** • pixels are useful for both horizontal and vertical dimensions
- `1px` = 1/96 of 1 inch, not relative
- corresponds to one device pixel
- high resolution (retina) displays `1px` may be several device pixels
- a pixel wide line is always sharp

100 vw = 100% of viewport width



100 vh = 100% of viewport height

Examples

- Default font-size (for the `<body>` element) in most browsers is 16px

- 1em = 16px
- 0.875em = 14px
- 0.75em = 12px

```
li {                                /* list items are written */
  font-size: 0.875em; /* smaller than its */
}

q {                                /* quoted text will be half */
  font-size: 0.5em; /* as big as its parent */
}
```

- Font size is one of the inherited properties

- children inherit the text size of their parent
- no need to repeat, unless text size is changed

- Which units to use?

- %-units for flexible width (changes with the window width)
- em or rem units for font-sizes
- px for fixed widths